COMPUTER PROGRAMMING

Name: Talha Ali
Registration No: BF24NWELE0728
Section: A (Power)
Electrical Department
UET Nowshehra

LAB 4

LAB TITLE

Switch Statement, Break and Continue in C++

OBJECTIVE

The objective of this lab is to understand the use of the switch statement in C++. Students also learn how the break statement controls program flow and how the continue statement is used inside loops. This lab also introduces simple menu-driven programs using switch statements.

2. THEORY

The switch statement **is used when a program needs to select one option from multiple choices based on the value of a variable. It is often used instead of multiple** if-else statements **when comparing a single variable with several constant values.**

Each option inside the switch statement is written as a case**. The program checks the value of the variable and executes the matching case block.**

The break statement **is used to stop execution inside a case. Without break, the program would continue executing the next cases.**

The default statement **is optional and runs when none of the case values match the given input.**

The continue statement **is used inside loops to skip the current iteration and move to the next iteration.**

3. SOFTWARE

- DEV-C++ IDE
- C++ Compiler (included in DEV-C++)

PROGRAM 1

Day Name Using Switch

Algorithm

- Start the program
- Declare an integer variable for day number
- Ask the user to enter a number from 1 to 7
- Use a switch statement to check the number
- Display the corresponding day name
- End the program

```
#include <iostream>
using namespace std;

int main() {
   int day;
   cout << "Enter day number (1-7): ";
   cin >> day;

   switch(day)
   {
      case 1: cout << "Monday"; break;
      case 2: cout << "Tuesday"; break;
      case 3: cout << "Wednesday"; break;
      case 4: cout << "Thursday"; break;
      case 5: cout << "Friday"; break;
      case 6: cout << "Saturday"; break;
      case 7: cout << "Sunday"; break;
      default: cout << "Invalid day number";
   }
  return 0;
}
```

PROGRAM 2

Grade Evaluation Using Switch

Algorithm

- **Start the program**
- **Declare a character variable for grade**
- **Ask the user to enter a grade**
- **Use a switch statement to evaluate the grade**
- **Display the corresponding message**
- **End the program**

```
#include <iostream>
using namespace std;
  int main()
{
        char grade;
             cout << "Enter your grade (A, B, C, D, F): ";
             cin >> grade;
        grade=toupper(grade);
        switch(grade)
{
      case 'A':
      cout << "Excellent";
      break;
      case 'B';   cout << "Very Good";
      break;
      case 'C':
      cout << "Good";
      break;
      case 'D':
      cout << "Pass";
      break;
      case 'F':
      cout << "Fail";
      break;
      default:
      cout << "Invalid Grade";
}

   return 0;
  }
```

PROGRAM

Unit Conversion Menu Using Switch

Algorithm

- Start the program
- Display a menu of conversion options
- Take user choice
- Ask for input value
- Perform the selected conversion
- Display the result.

```
#include <iostream>
using namespace std;
int main()
{
        int choice;
       double value;
       cout << "1 → Kilometers to Meters\n";
        cout << "2 → Meters to Centimeters\n";
        cout << "3 → Kilograms to Grams\n";
        cout << "4 → Celsius to Fahrenheit\n";
        cout << "Enter your choice: ";
        cin >> choice;
        cout << "Enter value: ";
        cin >> value;
        switch(choice)
{
         case 1:
cout << "Meters = " << value * 1000;
break;
         case 2:
cout << "Centimeters = " << value * 100;
break;
         case 3:
cout << "Grams = " << value * 1000;
  break;
         case 4:
  cout << "Fahrenheit = " << (value * 9/5) + 32;
  break;
         default:
   cout << "Invalid choice";
```

```
    }
    return 0;
    }
```

Conclusion

In this lab we learned how to use the **switch statement** for multiple decision making in C++. We also understood the importance of the **break statement** in controlling program flow. This lab helped us develop simple menu-based programs and improved our understanding of conditional structures.